

Lecture 3: Computer Vision

41-501
Applied AI

etienne@cmkl.ac.th

Content

- Intro
- CNNs
- Vision Tasks
- Transfer Learning
- Data
- Outlook
- Lab



What is "Computer Vision"?

- Extracting structured information from images or video
- Input: pixels (a grid of numbers)
- Output: labels, bounding boxes, masks, embeddings...
- ”A matrix of RGB values" → "a cat sitting on a chair" = CV



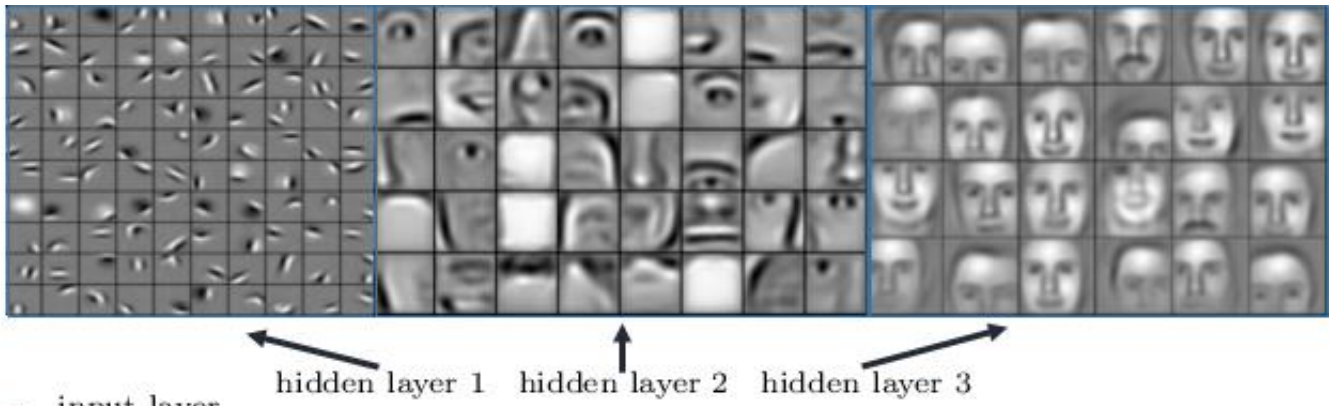
Why is computer vision hard?

- Same object looks completely different under different conditions
- Viewpoint changes, lighting, occlusion, deformation, background clutter
- A grayscale 28×28 image (MNIST) = 784 inputs (manageable)
- A color 224×224 image (ImageNet) = 150,528 inputs
- First hidden layer with 1,000 neurons → 150 million weights
- That's only ONE layer (no training yet).
- In practice: unfeasible memory, slow training, severe overfitting
- Needs a smarter approach!

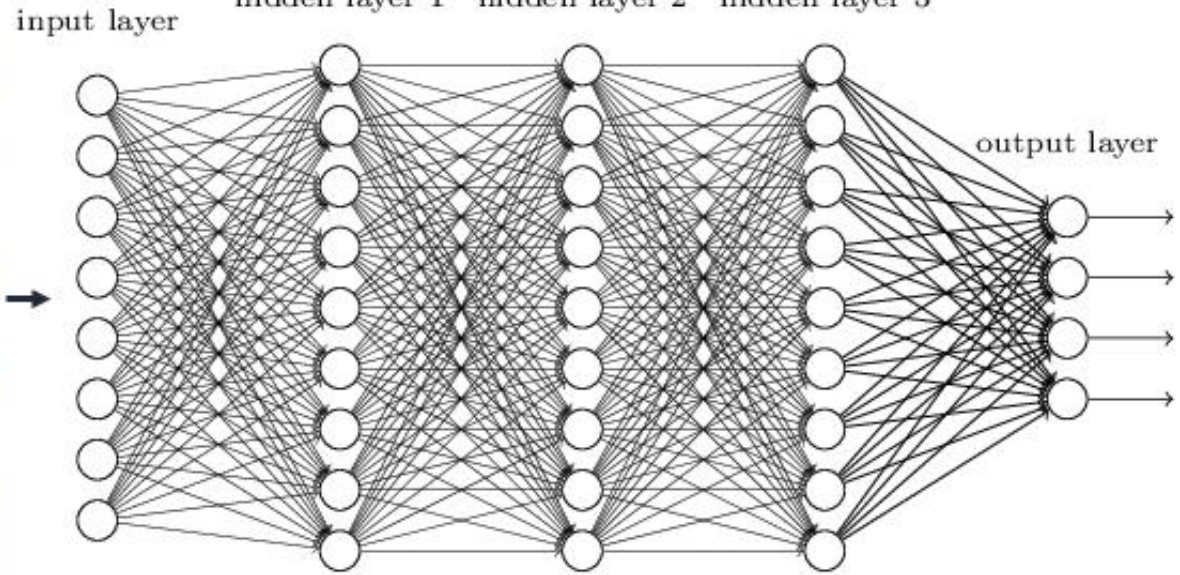


Why is computer vision hard?

Deep neural networks learn hierarchical feature representations







What makes images special?

- Pixels are not independent: nearby pixels are strongly correlated
- Features are local: an edge, a corner, a texture lives in a small region
- Features are reusable: the same edge detector is useful everywhere in the image
- A fully connected layer treats pixel (0,0) and pixel (200,200) as equally related (wrong)

Three properties we want to exploit:

1. Local connectivity: features depend on small patches, not the whole image
2. Weight sharing: use the same detector at every position
3. Translation invariance: a cat in the top-left and bottom-right should activate the same feature

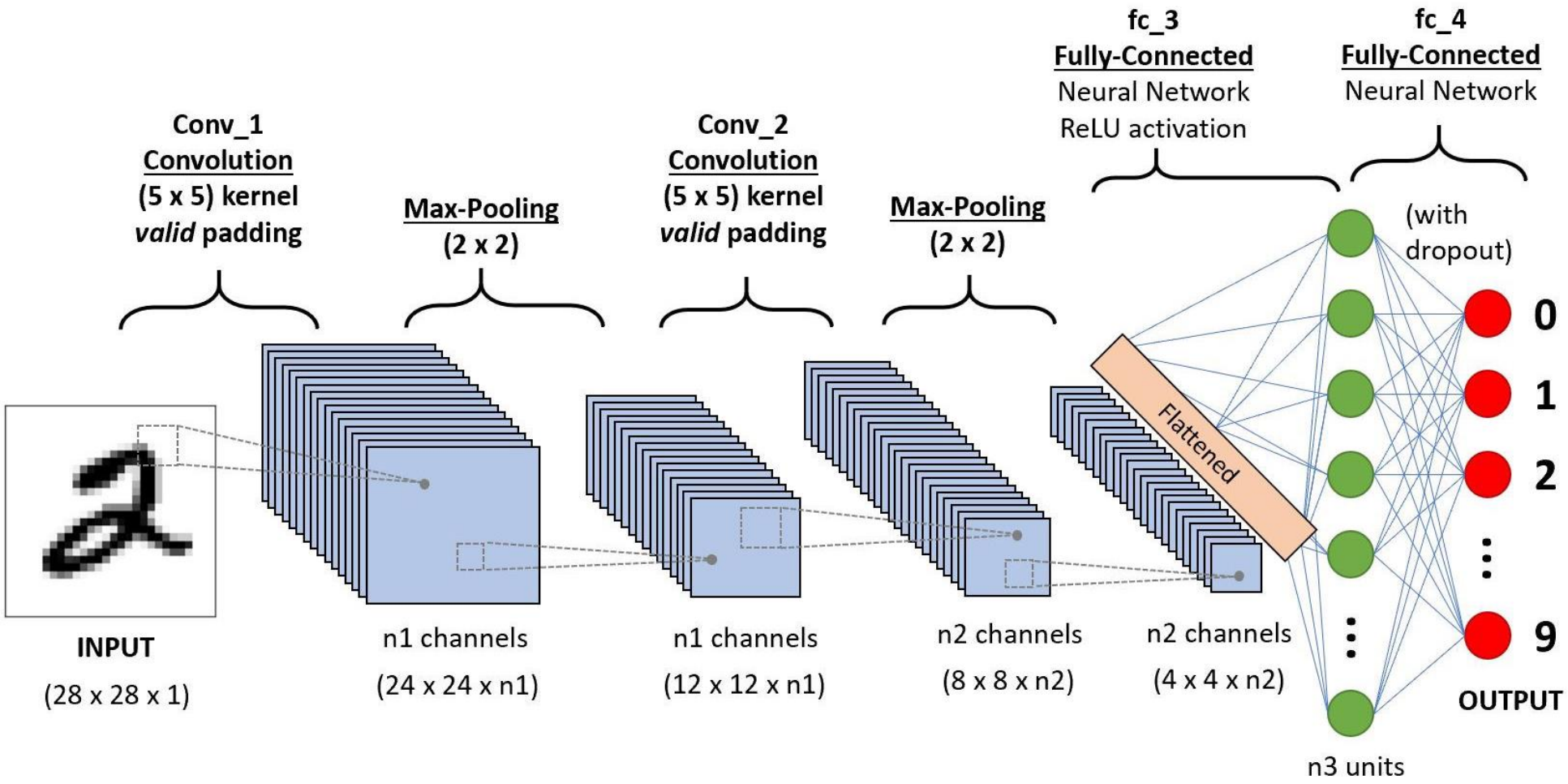


Content

- Intro
- CNNs
- Vision Tasks
- Transfer Learning
- Data
- Outlook
- Lab

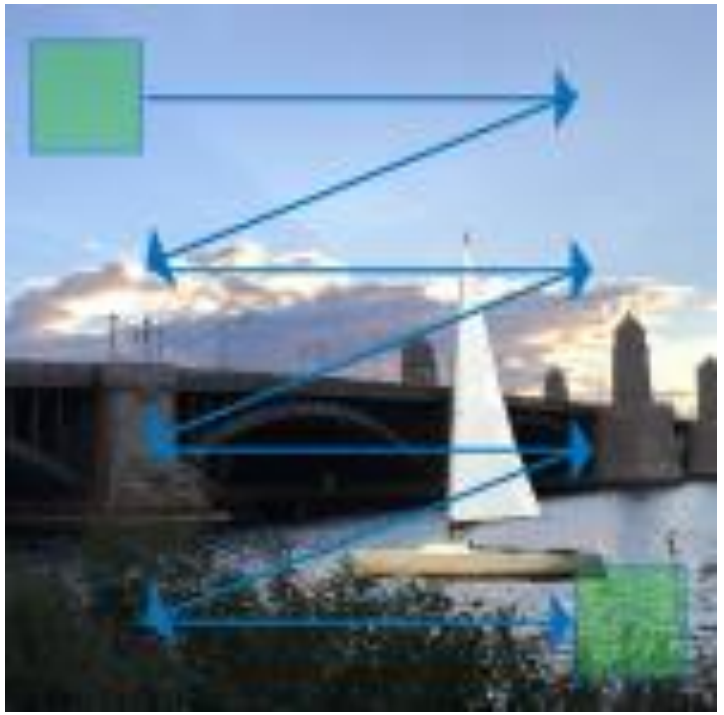


Convolutional Neural Networks (CNNs)



Filter/Kernel

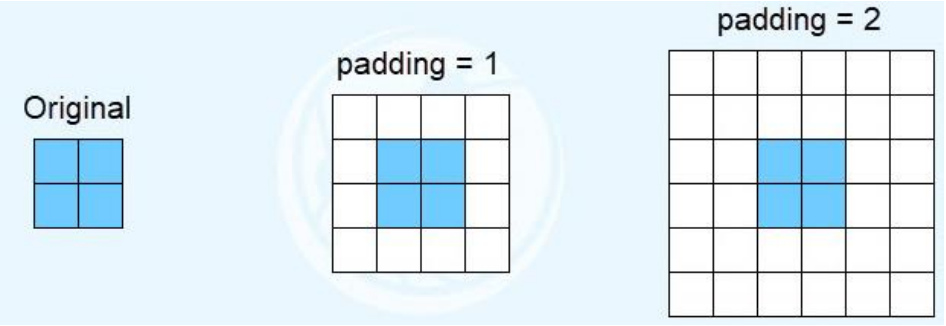
- Small 3×3 "edge detector"
- Slide it across the image
- Convolution
- Results in a feature map: high values where the feature was found, low values elsewhere
- Key difference from an MLP: the same weights are reused at every position





Padding

- Without padding, each convolution shrinks the spatial dimensions
- 10 conv layers without padding on 32×32 image → 0
- Valid padding (P=0): no padding, output is smaller than input
- Same padding: pad with zeros so output spatial size = input spatial size
  - For F=3: P=1. For F=5: P=2. General rule: P = (F−1)/2
  - Same padding is the default in most modern architectures
- It also ensures edge pixels receive as much "attention" as center pixels

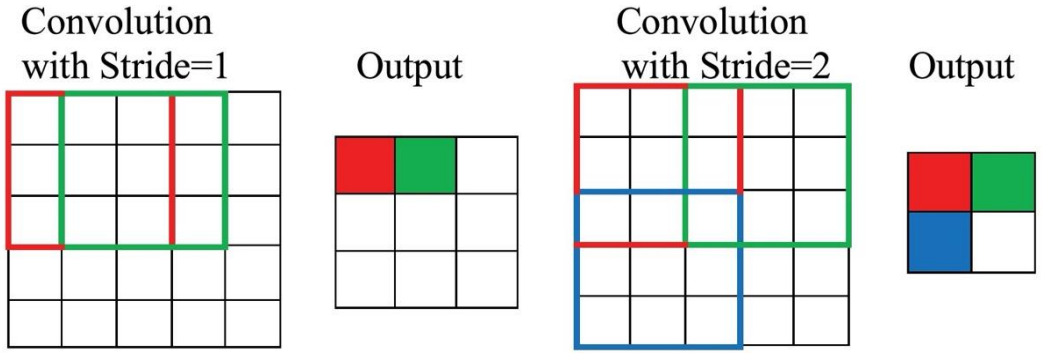




Stride

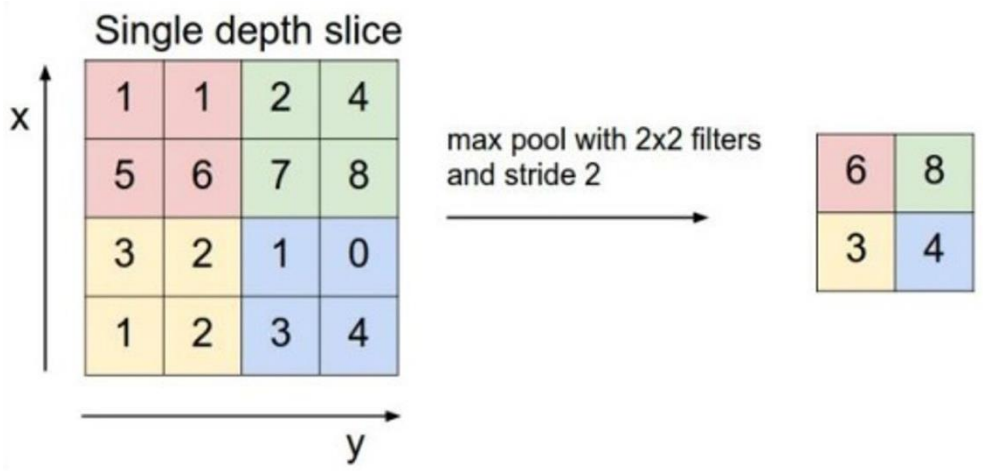
- Stride = how many pixels the filter moves at each step
- Stride 1: dense, overlapping: output nearly same size as input
- Stride 2: output is roughly half the spatial dimensions
- Stride > 1 is an alternative to pooling for downsampling
- Higher stride → fewer output values

Modern architectures (e.g. ResNet) often prefer stride-2 convolutions



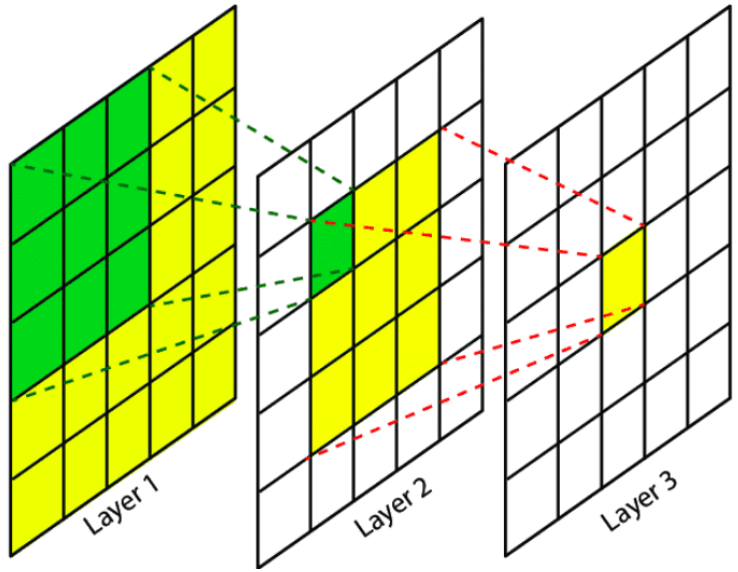
Pooling layers

- Pooling is a spatial downsampling operation: no learnable parameters
- Applied independently to each feature map (depth unchanged)
- Max pooling: takes the maximum value in each local window
  - "Was this feature present anywhere in this region?"
  - Retains the strongest activation
- Average pooling: takes the mean
  - Smoother summary of the region
  - Common as a final global operation before classification
- Typical: 2×2 window, stride 2
  → halves both spatial dimensions



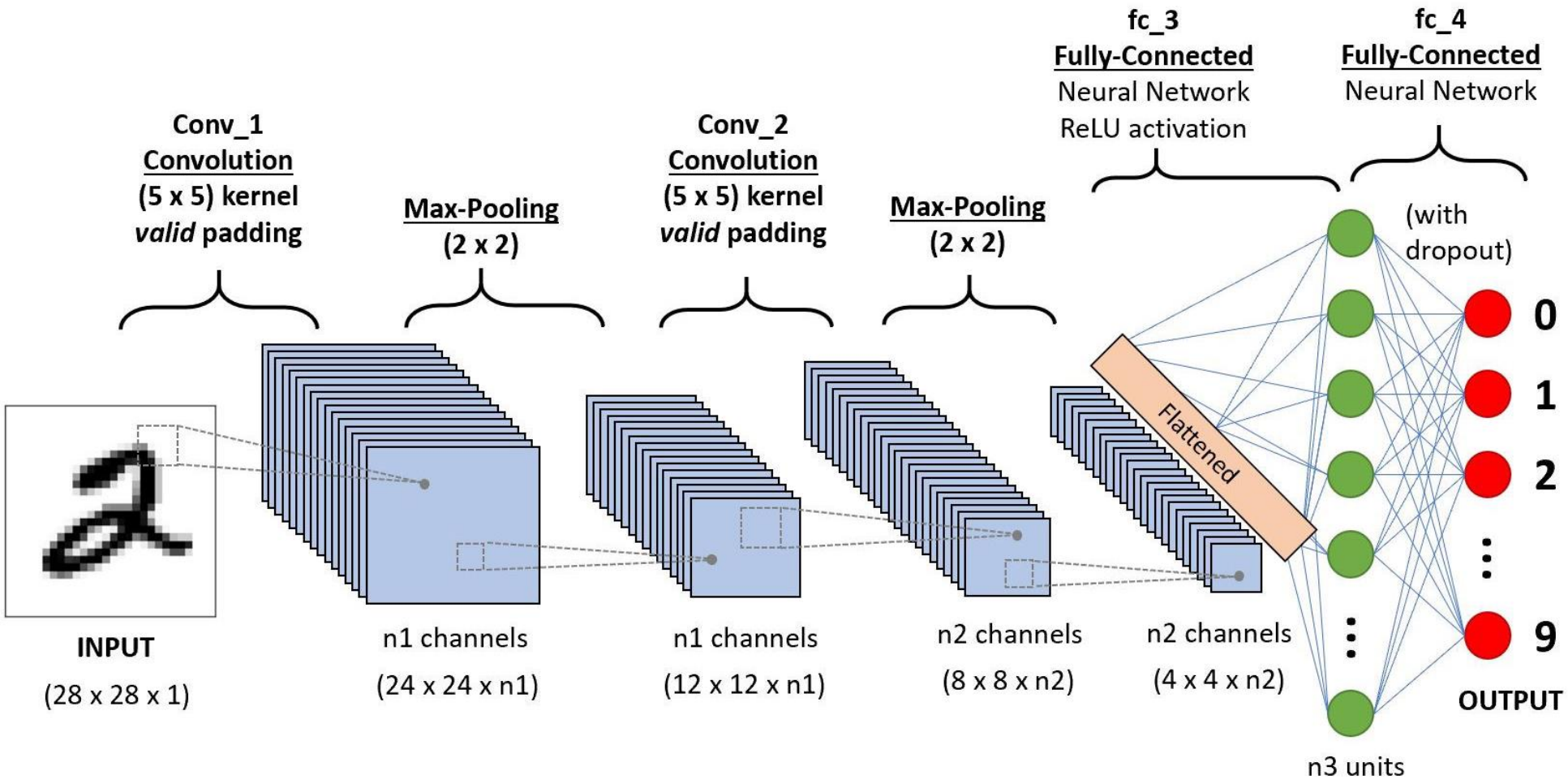
Receptive field

- The **receptive field** of a neuron = the region of the original input that influences its activation
- A single 3×3 conv → receptive field of 3×3
- Two stacked 3×3 convs → 5×5 receptive field
- Three stacked → 7×7
- Each max pool 2×2 doubles the receptive field of everything after it





Convolutional Neural Networks (CNNs)



Parameter count

Fully connected layer: mapping (32×32×3 = 3,072 values) to 32 neurons:

- Total = 3,072 × 32 + 32 = 98,336 parameters

Conv layer with 32 filters of size 3×3 on a 3-channel input:

- Parameters per filter = 3 × 3 × 3 + 1 (bias) = 28
- Total = 28 × 32 = 896 parameters

Conv uses ~110× fewer parameters for the same task

This comes entirely from *weight sharing*: the same filter is reused at every spatial position rather than each position having its own weights.



Key architectures

Year	Architecture	Key contribution
1998	LeNet-5	First successful CNN (digit recognition)
2012	AlexNet	“Deep” CNN wins ImageNet (ReLU and dropout, GPU)
2014	VGGNet	All 3×3 filters; showed depth matters
2015	GoogLeNet	Inception modules; 1×1 convolutions
2015	ResNet	Residual connections; trained 152-layer networks
2017+	MobileNet, EfficientNet	Efficient CNNs for deployment

Each architecture solved a problem the previous one couldn’t.
AlexNet (8 layers) to ResNet (152 layers): made possible by residual connections.



Content

- Intro
- CNNs
- Vision Tasks
- Transfer Learning
- Data
- Outlook
- Lab



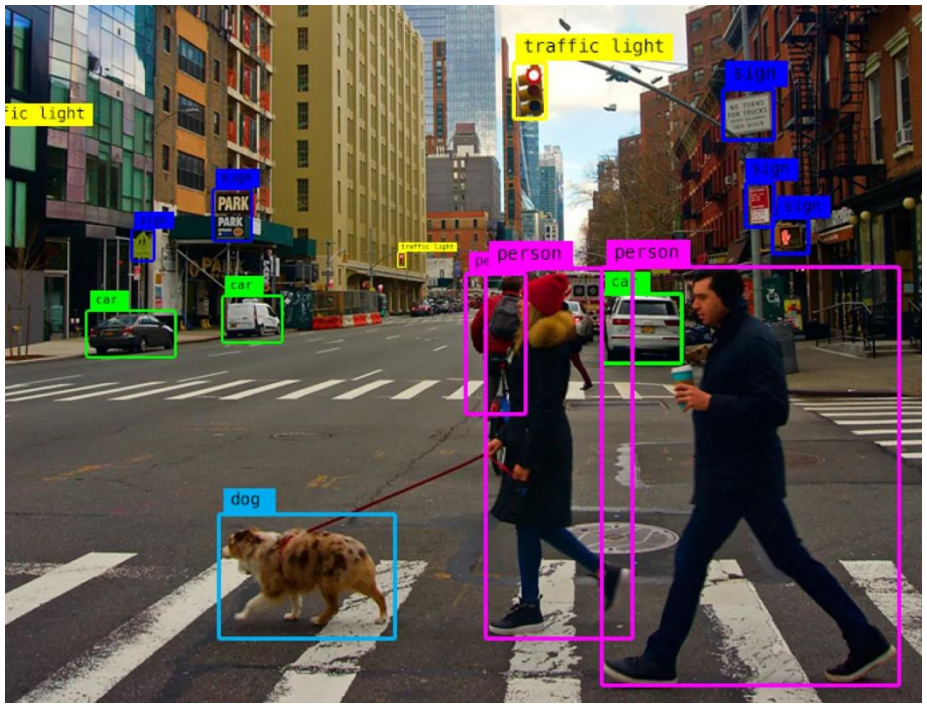
Task 1: Image Classification

- Input: an image → Output: a label (or probability distribution over labels)
- "This is a cat" / "This X-ray shows pneumonia" / "This product is defective"
- The most fundamental vision task
- Single-label vs multi-label



Task 2: Object Detection

- Input: an image → Output: bounding boxes + labels
- "There's a car here, a person there, a stop sign there"
- Applications: autonomous driving, surveillance, retail analytics, medical imaging
- Key models: YOLO family, Faster R-CNN, DETR



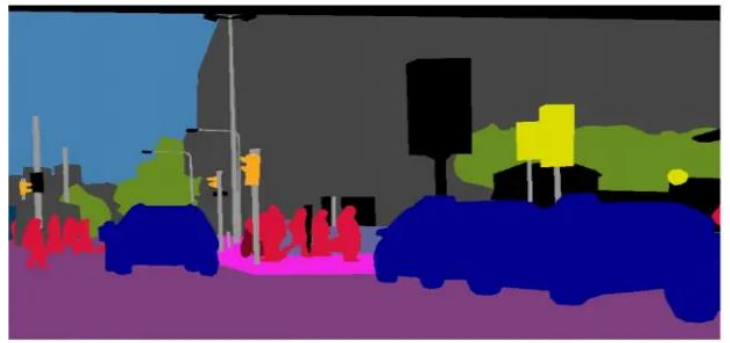


Task 3: Semantic Segmentation

- Input: an image → Output: a class label for every pixel
- "This pixel is road, this pixel is sidewalk, this pixel is sky"
- No distinction between instances: all "person" pixels are the same class
- Applications: autonomous driving, medical imaging, satellite imagery



(a) image

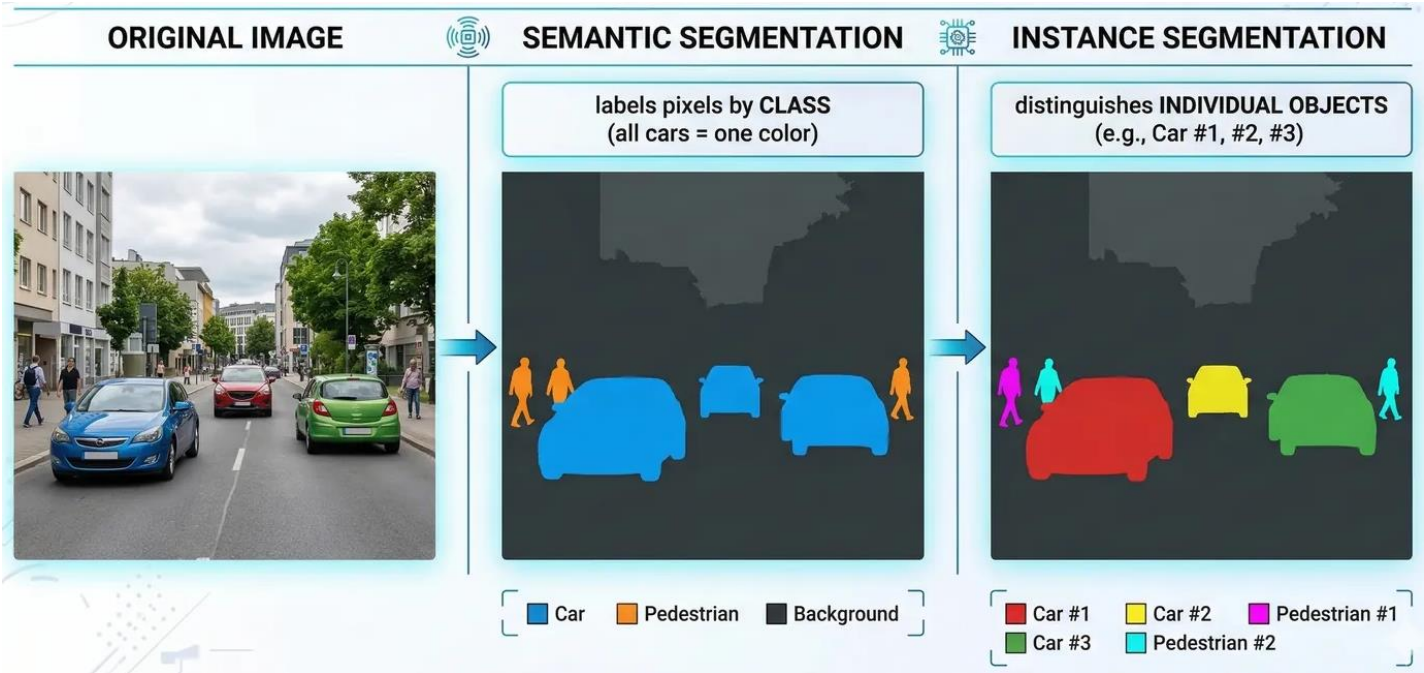


(b) semantic segmentation



Task 4: Instance Segmentation

- Like semantic segmentation but distinguishes individual objects
- Car 1, car 2: each gets their own mask
- Combines detection (which objects?) with segmentation (exact shape)
- Key model: Mask R-CNN



Task 5: Other vision tasks

- Pose estimation: keypoints on human body (skeleton tracking)
- Image generation: GANs, diffusion models (DALL-E, Stable Diffusion)
- Depth estimation: monocular or stereo
- OCR / Document analysis: text extraction from images
- Video understanding: action recognition, tracking



Content

- Intro
- CNNs
- Vision Tasks
- Transfer Learning
- Data
- Outlook
- Lab



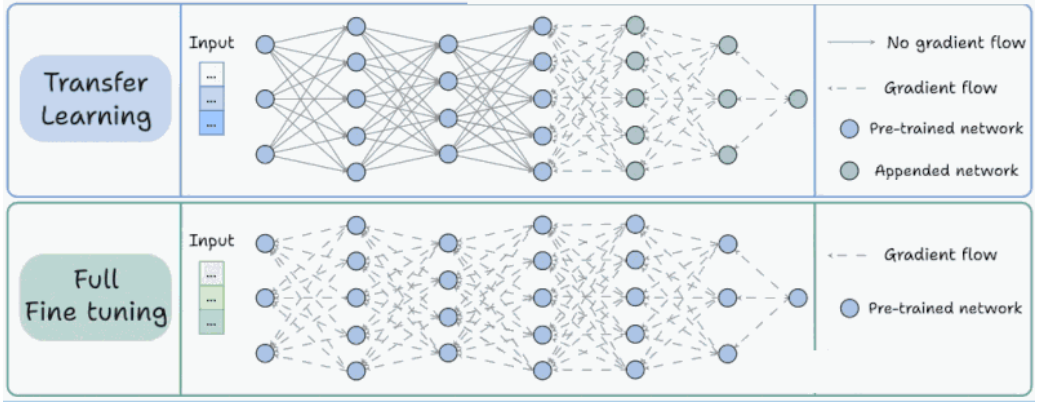
The data problem

- Custom project: maybe ~500 labeled images
- Training CNN on only 500 images → overfitting
- Add data, e.g. ImageNet (1.2 million labeled images)
- Training a CNN still requires significant compute (days/weeks on GPUs)
- Solution: start from already learned features



Transfer learning

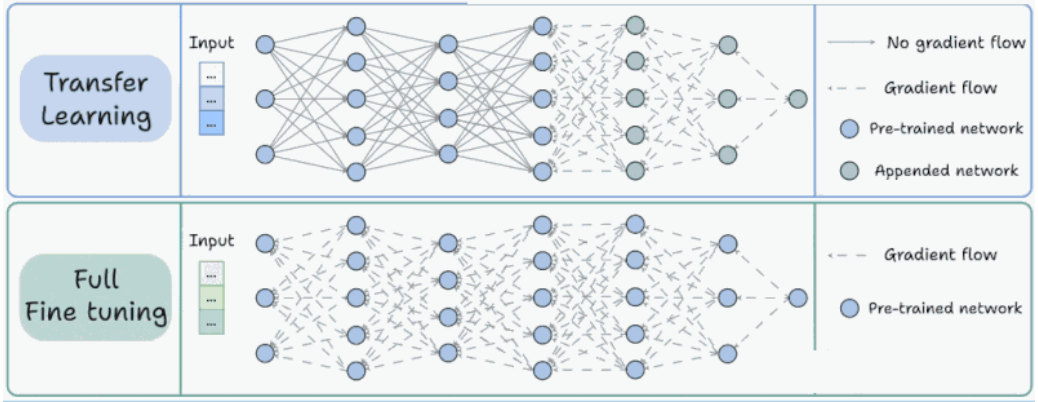
- **Transfer learning:** take a trained CNN, use its weights as a starting point
- Two approaches:
  - **Feature extraction:** freeze conv layers, only train the classifier head on your new task
  - **Fine-tuning:** start from pretrained weights, continue training all (or some) layers on your data
- In practice: almost always better to start from a pretrained





Transfer learning: Approach

- **Feature extraction**: freeze all pretrained layers, only train the new head: fast, works with very little data
- **Fine-tune top layers**: unfreeze the last few layers: more flexibility
- **Full fine-tuning**: unfreeze everything, train with low learning rate: best performance, needs more data
- **Rule of thumb**: less data → freeze more



Choosing a pretrained backbone

- ResNet-50: solid default, well-understood, fast
- EfficientNet: better accuracy/efficiency tradeoff
- ConvNeXt: modern CNN, competitive with transformers
- ViT (Vision Transformer): state-of-the-art but needs more data
- In practice: ResNet-50 or EfficientNet, upgrade if needed



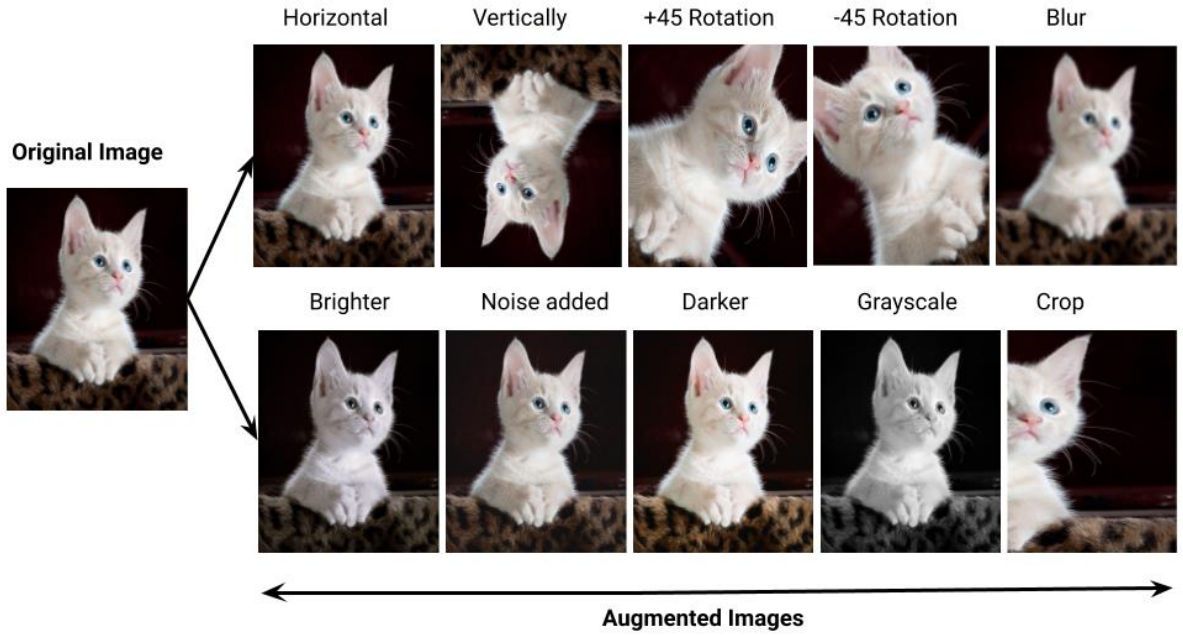
Content

- Intro
- CNNs
- Vision Tasks
- Transfer Learning
- Data
- Outlook
- Lab



Data augmentation

- Artificially expand the training set by applying transformations:
  - Geometric: rotation, flipping, cropping, scaling
  - Photometric: brightness, contrast, color jitter, blur
  - Advanced: random erasing, Mixup, CutMix
- Free performance boost: always use augmentation



Handling small datasets

- Transfer learning: strategy number one
- Heavy data augmentation
- Collect more data
- Semi-supervised learning: use unlabeled data
- Synthetic data: render 3D models, use generative AI
- Bonus: Know when to stop



Common pitfalls

- Data leakage: your test set contains images too similar to training (same video, same patient, same session)
- Class imbalance: 95% negative, 5% positive → model predicts "negative" always and gets 95% accuracy
- Evaluation on the wrong metric: accuracy is misleading when classes are imbalanced: use precision, recall, F1, mAP
- Train/test distribution mismatch: training on stock photos, deploying on phone camera images



Content

- Intro
- CNNs
- Vision Tasks
- Transfer Learning
- Data
- Outlook
- Lab



Vision Transformers (ViT)

- Apply the transformer architecture directly to image patches
- Split image into 16×16 patches, treat each patch like a "token"
- Similar or even better than CNNs at scale
- Need more data to train from scratch, but pretrained ViTs work well with transfer learning
- Line between "vision model" and "language model" is blurring



Foundation models: CLIP

- Trained on 400M image-text pairs
- Learns a shared embedding space: images and text in the same vector space
- Zero-shot classification: describe categories in text, no training data needed
- "A photo of a cat" vs "A photo of a dog”: CLIP picks the closer match

- → Multimodal AI: models that handle images, text, audio, video together



Summary

- CNNs learn hierarchical features automatically
- Tasks: classification → detection → segmentation (increasing complexity and data needs)
- Transfer learning is the default
- Data quality and augmentation matter more than architecture tweaks
- Field is moving toward foundation models and multimodal systems



Content

- Intro
- CNNs
- Vision Tasks
- Transfer Learning
- Data
- Outlook
- Lab



ขอบคุณครับ

Thank you!

etienne@cmkl.ac.th

